

Lezione 3 - Progettazione Dei Microservizi

Progettazione dei Microservizi — Dal "Come" al "Cosa"

Finora abbiamo esplorato l'architettura a microservizi da un punto di vista **infrastrutturale**: abbiamo visto come si organizzano i componenti all'interno del Boundary, come comunicano attraverso il Gateway, come si registrano su Eureka, come si configurano centralmente.

Abbiamo imparato **come si fa a far parlare i microservizi tra loro** e **come si costruisce la "scatola" che li contiene**.

Ma c'è una domanda che abbiamo lasciato in sospeso, e che ora dobbiamo affrontare: **cosa mettiamo dentro quella scatola?**

Progettare un'applicazione a microservizi non significa solo scegliere gli strumenti giusti (Spring Cloud, Eureka, Kafka) e disegnare la rete.

Il vero problema — quello più profondo e meno tecnico — è **architetturale** nel senso più nobile del termine:

1. **Come divido le responsabilità? — Quali sono i confini logici tra un microservizio e l'altro? Dove finisce il "Catalogo" e dove inizia "Ordini"? Se sbaglio questa divisione, mi ritrovo con un "distributed monolith" — il peggio di entrambi i mondi.**
2. **Come gestisco i vari database? — Ogni microservizio ha il suo database dedicato, ma questo significa che una transazione che coinvolge più servizi (ad esempio, creare un ordine e aggiornare il catalogo) non può più essere una semplice transazione ACID. Come garantiamo la consistenza?**
3. **Quale tecnica uso per far comunicare i servizi tra loro? — Chiamate REST sincrone? Code asincrone? Eventi in broadcast? La scelta non è solo tecnica — ha implicazioni profonde su resilienza, consistenza e accoppiamento tra i servizi.**

Per capire come affrontare queste domande, niente è meglio di un esempio concreto. Prendiamo un'applicazione di **Food Delivery** — come Glovo, Deliveroo o Just Eat — e proviamo a progettare la sua architettura a microservizi. Non partiamo dai componenti tecnici (Gateway, Registry, Config), ma dalle **capacità di business** dell'azienda e dai **domini di dati** che la attraversano.

Esempio Pratico: Food Delivery (come Glovo)

Per dividere correttamente un sistema in microservizi, bisogna guardare a due dimensioni fondamentali:

- **Le capacità di business** — cosa fa l'azienda? Quali sono le attività principali che generano valore?
- **I domini di dati** — quali dati tratta l'azienda? Quanto sono critici? Chi li possiede?

Il principio guida è sempre lo stesso:

- **ogni microservizio deve possedere i propri dati ed essere indipendente.**

Se due servizi condividono lo stesso database, non sono veramente indipendenti — **sono solo due facciate sulla stessa base dati, e una modifica a una tabella rischia di rompere entrambi.**

Ecco come si potrebbe scomporre un'applicazione di Food Delivery in microservizi:

A. Servizio Catalogo (Menu Service)

- **Responsabilità:** **Gestire i ristoranti, i piatti, i prezzi e la disponibilità (es. "terminato", "disponibile solo fino alle 15:00").**
- **Autonomia:** **I ristoratori devono poter aggiornare il proprio menu in qualsiasi momento, senza dover coinvolgere il resto del sistema. Questo servizio è il loro punto di accesso.**

Nota:

Potrebbe essere ulteriormente scomposto secondo il pattern **CQRS (Command Query Responsibility Segregation)** — ad esempio, un `catalog-rd` per le letture (query) e un `catalog-wr` per le scritture (command), ottimizzati separatamente.

B. Servizio Ordini (Order Service)

- **Responsabilità:** **Creazione dell'ordine, calcolo del totale, gestione dello stato dell'ordine (ricevuto, in preparazione, pronto, consegnato).**

- **Autonomia:** È il cuore del processo operativo. Deve poter evolvere indipendentemente da come sono strutturati i menu o da come vengono gestiti i pagamenti.

C. Servizio Pagamenti (Payment Service)

- **Responsabilità:** Interfacciarsi con gateway di pagamento esterni (Stripe, PayPal, circuiti carte) e gestire le ricevute.
- **Autonomia:**
 - È il servizio più critico per la sicurezza.
 - Isolarlo in un microservizio dedicato permette di restringere il perimetro di conformità PCI-DSS (Payment Card Industry Data Security Standard),
 - le norme di sicurezza per chi gestisce dati di carte di credito.
 - Più piccolo è il perimetro, più semplice è la certificazione.

D. Servizio Logistica/Rider (Delivery Service)

- **Responsabilità:** Localizzazione GPS dei rider, assegnazione della consegna al rider più vicino, calcolo dei tempi di arrivo stimati.
- **Autonomia:**
 - Questo servizio ha requisiti di latenza molto stringenti (tracciamento in tempo reale) e una logica di ottimizzazione complessa.
 - Separarlo permette di scalarlo indipendentemente dagli altri.

Divisione delle Responsabilità — Il Pattern CQRS

Nell'esempio del Food Delivery, abbiamo accennato alla possibilità di scomporre il servizio Catalogo in due componenti distinti secondo il pattern **CQRS** (Command Query Responsibility Segregation). Ora approfondiamo questa scelta architetturale, perché tocca un punto fondamentale nella progettazione dei microservizi.

Considerazione N.1: Lettura e Scrittura Hanno Carichi Diversi

Una verità spesso sottovalutata nella progettazione è questa:

I servizi di scrittura e lettura non hanno solitamente la stessa frequenza d'uso, né lo stesso carico di lavoro.

Pensiamo al catalogo prodotti di Glovo o Deliveroo:

- **Letture** — ogni secondo, migliaia di utenti consultano il menu, cercano ristoranti, filtrano per cucina, leggono i prezzi.
- **Scritture** — un ristorante modifica il proprio menu, aggiunge un piatto, aggiorna la disponibilità. Questo accade con frequenza molto minore.

Se mettiamo lettura e scrittura nello stesso microservizio, con lo stesso database, ci troviamo con un unico componente che:

- Deve gestire picchi di lettura enormi
- Deve garantire consistenza sulle scritture
- Scala in modo uniforme per entrambi i carichi, anche se hanno esigenze diverse

La Soluzione: Due Microservizi Indipendenti

Una soluzione architetturale valida è **separare le responsabilità**:

Microservizio	Responsabilità	Frequenza	Caratteristiche
catalogo_WR	Operazioni di scrittura (inserimento, modifica, cancellazione prodotti)	Bassa	Deve garantire consistenza, validazione, transazioni
catalogo_RD	Operazioni di lettura (consultazione catalogo, ricerca, filtri)	Alta	Ottimizzato per performance, può denormalizzare i dati, caching aggressivo

Questi due microservizi **sono indipendenti**:

- Possono essere scalati separatamente — se il catalogo viene consultato da milioni di utenti, scaliamo catalogo_RD ; se pochi ristoranti aggiornano i menu, catalogo_WR rimane piccolo.
- Possono essere sviluppati e deployati indipendentemente
- Possono utilizzare tecnologie diverse (es. catalogo_RD potrebbe usare Elasticsearch per ricerche full-text, catalogo_WR un database

relazionale)

👁 Il pattern CQRS:

Questo approccio è noto come **CQRS (Command Query Responsibility Segregation)** — **separazione delle responsabilità tra comandi (scrittura) e query (lettura).**

Non è obbligatorio in ogni microservizio, ma diventa prezioso quando i carichi di lettura e scrittura sono molto sbilanciati.

Gestione dei Database — Master e Replica

Separare lettura e scrittura in due microservizi risolve il problema della scalabilità, ma ne solleva un altro: **i dati come vengono condivisi?**

Considerazione N.2: Database Propri ma Dati Condivisi

Abbiamo stabilito che ogni microservizio ha il proprio database — questo è un principio fondamentale.

Ma catalogo_WR **e** catalogo_RD **operano sugli stessi dati (i prodotti, i menu, i prezzi).**

Come gestiamo questa condivisione senza violare il principio di autonomia?

La Soluzione: Master e Replica

La strategia è chiara:

Il microservizio che fa principalmente modifiche (scrittura) possiede il database Master.

Gli altri microservizi (lettura) possiedono delle copie — database Replica — meno strutturate, più leggere, che si sincronizzano con il Master all'occorrenza.

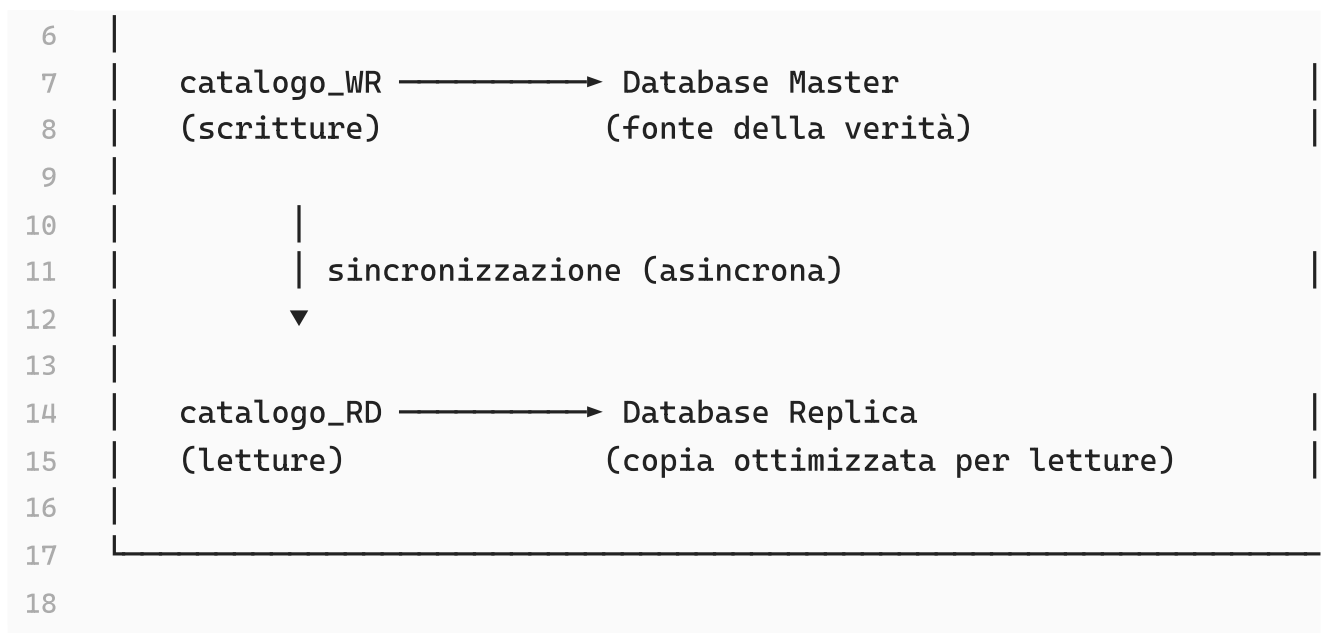
Nel nostro esempio:



Plain text

1
2
3
4
5

MICROSERVIZI



Database Master (di `catalogo_WR`):

- È la fonte della verità — contiene i dati completi e consistenti
- Gestisce transazioni ACID
- Riceve tutte le operazioni di scrittura

Database Replica (di `catalogo_RD`):

- È una copia del Master, sincronizzata asincronamente
- È ottimizzata per le letture — può essere denormalizzata, indicizzata diversamente, persino su tecnologie diverse (es. Elasticsearch, Redis)
- Non accetta scritture direttamente

Come Avviene la Sincronizzazione?

La replica può essere implementata in diversi modi:

1. Replica a livello di database:

- se entrambi i microservizi usano lo stesso tipo di database (es. entrambi PostgreSQL), si può configurare una replica nativa.
- Il database Master propaga le modifiche al database Replica automaticamente.

2. Event sourcing:

- `catalogo_WR` pubblica eventi ("prodotto creato", "prezzo aggiornato") su un sistema di messaggistica (Kafka, RabbitMQ).
- `catalogo_RD` consuma questi eventi e aggiorna la propria replica di conseguenza.

3. Sincronizzazione periodica:

- meno frequente, ma possibile per dati non critici in tempo reale.

✓ Vantaggi di Questo Approccio

- Isolamento dei fallimenti:
 - se il database Replica di `catalogo_RD` va giù, `catalogo_WR` continua a funzionare (e viceversa)
- Scalabilità indipendente:
 - puoi avere più repliche di lettura distribuite geograficamente, senza toccare il Master
- Prestazioni:
 - le letture non competono mai con le scritture per le stesse risorse
- Sicurezza:
 - il database Replica può esporre una vista ristretta dei dati, escludendo campi sensibili

In sintesi:

Quando un dominio di business ha carichi di lettura e scrittura molto sbilanciati:

1. Si applica il pattern **CQRS**, separando le responsabilità in due microservizi indipendenti
2. Si assegna il **database Master** al microservizio di scrittura
3. Si forniscono **database Replica** (uno o più) al microservizio di lettura
4. Si implementa un meccanismo di **sincronizzazione** (replica nativa o basata su eventi)

In questo modo si ottiene il meglio di entrambi i mondi: autonomia dei microservizi, scalabilità orizzontale, e consistenza dei dati.



Transazioni ACID (Atomicity, Consistency, Isolation, Durability)

Quando parliamo di database e microservizi, il termine **ACID** torna continuamente.

È uno di quei concetti fondamentali che devi avere ben chiari per capire perché i microservizi complicano la gestione dei dati.

ACID è un acronimo che descrive le **proprietà che una transazione su database deve garantire** per essere affidabile. Sta per:

Lettera	Proprietà	Significato
A	Atomicity (Atomicità)	O la transazione viene completata tutta, oppure non viene eseguita per niente. Non esiste uno stato intermedio.
C	Consistency (Consistenza)	La transazione porta il database da uno stato valido a un altro stato valido, rispettando tutte le regole (vincoli, foreign key, ecc.).
I	Isolation (Isolamento)	Transazioni concorrenti vengono eseguite come se fossero in sequenza, una dopo l'altra. Una transazione non vede gli stati intermedi di un'altra.
D	Durability (Durabilità)	Una volta che la transazione è stata confermata (commit), i dati rimangono persistenti anche in caso di crash del sistema.

Esempio pratico — Un Bonifico Bancario

Il modo migliore per capire ACID è con un esempio classico: **un bonifico di 100€ tra due conti correnti.**

Dati iniziali:

- **Conto di Alice: 200€**
- **Conto di Bob: 50€**

La transazione "trasferisci 100€ da Alice a Bob" richiede due operazioni:

1. Preleva 100€ da Alice → Alice diventa 100€
2. Deposita 100€ su Bob → Bob diventa 150€

Ecco come le proprietà ACID proteggono questa transazione.

1. Atomicità (A)

- Se il prelievo da Alice riesce ma il deposito su Bob fallisce (per un errore di rete, un crash, ecc.), l'intera transazione viene annullata.
- Il risultato? Alice torna a 200€, Bob resta a 50€. Nessun danno.
- Senza atomicità, ti saresti ritrovato con Alice=100€ e Bob=50€ — 100€ spariti nel nulla.

2. Consistenza (C)

- La consistenza si vede quando si tenta di violare una regola del database.

Supponiamo che il database abbia un vincolo: "Il saldo di un conto non può mai diventare negativo" (un CHECK constraint).

Ora esegui una transazione diversa: "trasferisci 300€ da Alice a Bob".

- Alice ha 200€
- Bob ha 50€

Cosa succede?

1. Prelievo da Alice: $200€ - 300€ = -100€ \rightarrow$ viola il vincolo $\text{saldo} \geq 0$

Qui interviene la Consistenza ACID:

- Il database rileva la violazione del vincolo.
- La transazione viene annullata (abortita).
- Alice rimane a 200€, Bob rimane a 50€.
- Il database non viene mai portato in uno stato inconsistente (con un saldo negativo).

3. Isolamento (I)

- Immagina che mentre fai il bonifico di 100€ da Alice a Bob, qualcun altro stia facendo un bonifico di 50€ da Alice a Carol.
- L'isolamento garantisce che le due transazioni non si "intralcino". Il risultato finale sarà come se fossero avvenute una dopo l'altra, indipendentemente da quale sia partita prima.
- Senza isolamento, potresti leggere uno stato intermedio (es. Alice=100€ prima che Bob venga aggiornato) e prendere decisioni sbagliate.

Durabilità (D)

- Una volta che il bonifico è completato e il database ha detto "commit", i dati sono salvi su disco.
- Se si stacca la corrente un millisecondo dopo, al riavvio troverai A=100€ e B=150€. La transazione non viene persa.

Perché ACID è Importante nei Microservizi?

Nei sistemi monolitici, con un unico database, ACID è **scontato**. Fai una transazione che tocca più tabelle, e il database garantisce tutto.

Nei microservizi, il problema è che ogni microservizio ha il proprio database indipendente. **Non esiste un "super-database" che possa garantire transazioni ACID che coinvolgono più servizi.**

Riprendiamo l'esempio del Food Delivery:



Plain text

- 1 Ordine creato → aggiorna catalogo (scorta) → elabora pagamento

Questo coinvolge:

- Database di order-service
- Database di catalog-service
- Database di payment-service

Una transazione ACID non è possibile attraverso tre database separati.

Se il pagamento fallisce dopo che hai già aggiornato la scorta, non puoi fare un semplice "rollback" — i database non parlano tra loro in quel modo.

Ecco perché nei microservizi si introducono:

- **Pattern Saga** — una sequenza di transazioni locali con compensazioni
- **Eventual consistency** — accettare che il sistema sia coerente "alla fine", non immediatamente
- **Code di messaggi** — per gestire operazioni asincrone senza bloccare

Ma questa è una lezione successiva.

Per ora, tieni a mente che **ACID** è il gold standard della consistenza dei dati, ma nei microservizi devi imparare a convivere con vincoli meno rigidi.

☰ Analogia Rapida

Pensa a ACID come a un **distributore automatico**:

- **Atomicità** — o esce lo snack e vengono scalati i soldi, o non succede nulla. Non puoi perdere i soldi senza ricevere lo snack.
- **Consistenza** — il distributore non può mai trovarsi con più snack fuori che soldi incassati (rispetta i suoi vincoli).
- **Isolamento** — se due persone comprano contemporaneamente l'ultimo snack, una sola riesce. Non possono "vedersi" a metà dell'operazione.
- **Durabilità** — una volta che lo snack è uscito, anche se si stacca la corrente, il distributore "ricorda" che i soldi sono stati scalati.

📅 In una frase:

ACID è l'insieme delle garanzie che un database ti dà quando fai una transazione.

Nei microservizi, queste garanzie non valgono più attraverso i confini dei servizi — ed è per questo che tutto diventa più complicato.

Comunicazione tra Servizi — Sincrona vs Asincrona

Abbiamo visto come dividere le responsabilità (CQRS) e come gestire i dati (Master/Replica). Ora affrontiamo la terza grande domanda della progettazione: **come fanno i microservizi a parlare tra loro?**

Considerazione N.3: Due Tecniche, Due Scelte

I microservizi hanno bisogno di comunicare tra loro — alcuni anche con i client esterni.

Esistono 2 tecniche principali, con caratteristiche molto diverse:

Modalità	Esempio	Caratteristiche
Sincrona	Un servizio chiama l' <u>endpoint REST</u> di un altro e attende la risposta	Semplice da implementare, ma crea accoppiamento temporale
Asincrona (broadcast)	Un servizio pubblica un evento su una coda (Kafka, RabbitMQ); altri servizi leggono quando possono	Disaccoppiamento forte, maggiore resilienza, ma più complessa

La Strategia: REST fuori, Code dentro

Una strategia comune e consolidata è questa:

Le chiamate esterne (client → Gateway → microservizi) usano chiamate REST.

Le comunicazioni interne tra microservizi usano entrambe le tecniche, ma si preferiscono le code a messaggi perché consentono una gestione asincrona e una maggiore indipendenza tra i servizi.

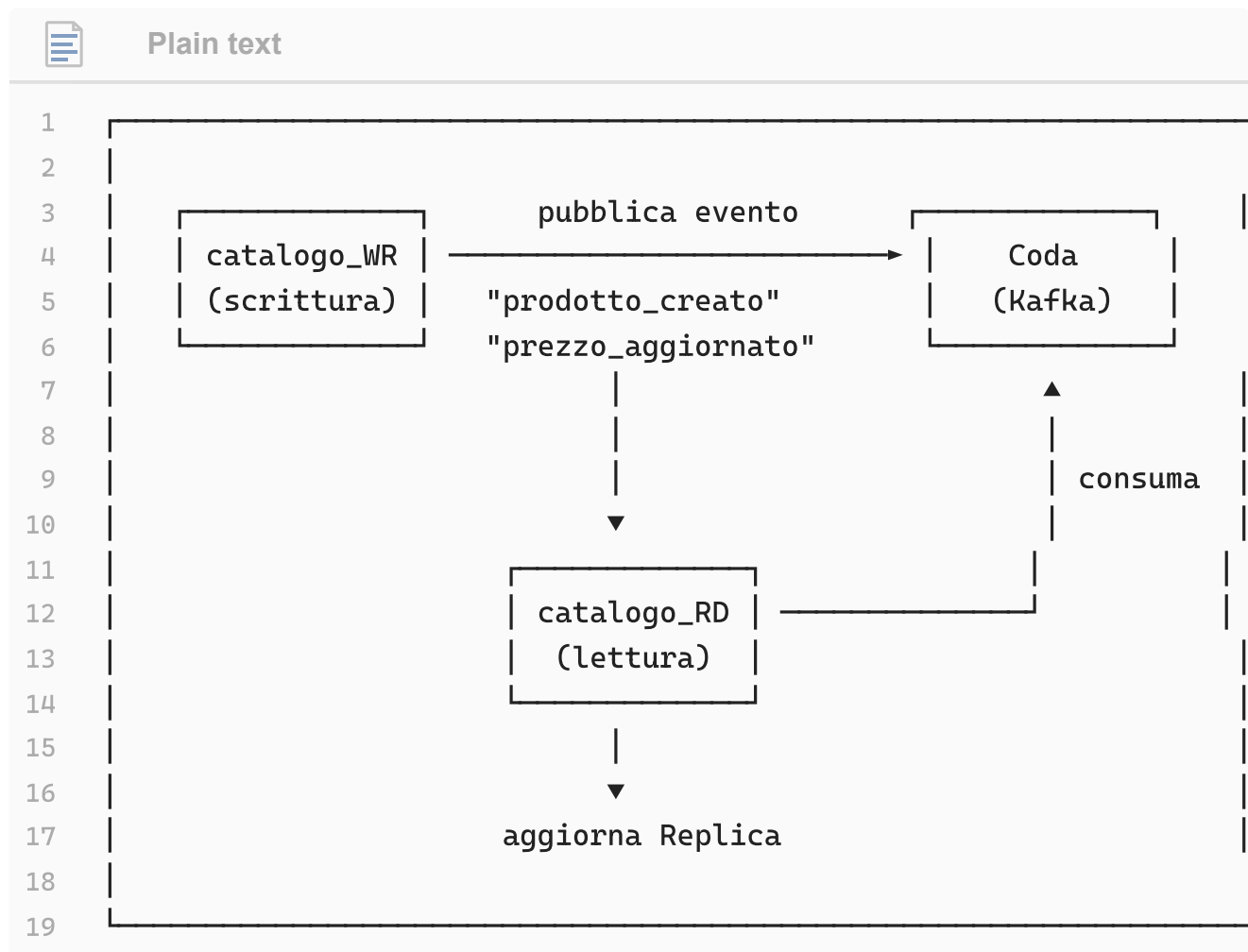
Perché questa distinzione?

- **REST per le chiamate esterne:**
 - i client (app mobile, browser, app desktop) si aspettano una risposta immediata.
 - Una chiamata REST con richiesta-risposta è il modello più naturale per loro.
- **Code per le comunicazioni interne** — quando un microservizio deve notificare un evento a un altro servizio, una coda asincrona permette:
 - **Disaccoppiamento temporale** — se il servizio destinatario è momentaneamente offline, il messaggio rimane nella coda e verrà processato al suo rientro
 - **Disaccoppiamento spaziale** — il servizio mittente non ha bisogno di sapere dove si trova il destinatario; pubblica su una coda e "se ne dimentica"

- **Broadcast naturale** — un solo messaggio può essere letto da più consumatori (es. `catalogo_WR` pubblica un evento `"prodotto_aggiornato"` che viene letto sia da `catalogo_RD` che dal servizio di ricerca)

Esempio: La Gestione del Catalogo con CQRS

Ritorniamo al nostro esempio di Food Delivery con `catalogo_WR` e `catalogo_RD`. Come comunicano?



Il flusso è:

1. Un ristorante modifica il prezzo di un piatto. La richiesta arriva a `catalogo_WR`.
2. `catalogo_WR` aggiorna il database Master.
3. `catalogo_WR` pubblica un evento su una coda (Kafka/RabbitMQ): `"prezzo_aggiornato"` con i dettagli del prodotto.
4. `catalogo_RD` consuma l'evento dalla coda (quando ha capacità di processarlo).
5. `catalogo_RD` aggiorna il database Replica di conseguenza.

✓ Vantaggi di Questo Approccio

- **Resilienza:**
 - se `catalogo_RD` è temporaneamente in difficoltà o in fase di deploy, gli eventi si accumulano nella coda.
 - Nessun dato va perso, e `catalogo_WR` continua a funzionare normalmente.
- **Scalabilità:**
 - se il carico di lettura aumenta, possiamo avviare più istanze di `catalogo_RD`, tutte che consumano dalla stessa coda.
 - Kafka bilancia automaticamente il carico tra i consumatori.
- **Disaccoppiamento:**
 - `catalogo_WR` non ha bisogno di sapere chi sono i consumatori degli eventi. Se domani aggiungiamo un nuovo servizio (es. `search-indexer` che aggiorna Elasticsearch), basterà farlo consumare dalla stessa coda — `catalogo_WR` non deve cambiare nulla.

Quando Usare REST tra Microservizi?

Le code non sono sempre la soluzione migliore. Ci sono casi in cui la comunicazione sincrona via REST è più appropriata:

- Quando è necessaria una risposta immediata — se un servizio deve sapere il risultato di un'operazione prima di procedere
- Quando l'operazione è poco frequente e non critica per la resilienza
- Per semplicità — in domini piccoli, introdurre una coda potrebbe essere over-engineering

Regola pratica:

- Se un servizio deve sapere con certezza che qualcosa è successo prima di continuare → usa REST
- Se un servizio deve notificare che qualcosa è successo, senza aspettarsi una risposta → usa una coda
- Se più servizi devono reagire allo stesso evento → usa una coda (broadcast)

Riepilogo delle Scelte di Comunicazione

Scenario	Tecnica	Motivo
Client → Gateway → microservizio	REST	Il client si aspetta una risposta immediata
catalogo_WR → catalogo_RD	Coda	Aggiornamento asincrono della replica
order-service → payment-service	REST	Devo sapere se il pagamento è riuscito prima di confermare l'ordine
order-service → delivery-service	Coda	Devo solo notificare che un ordine è pronto per la consegna
order-service → notification-service	Coda	Invia email/SMS di conferma — nessuna risposta necessaria

In sintesi:

La scelta tra REST e code non è una decisione binaria da prendere una volta per tutte. Un'applicazione a microservizi ben progettata **utilizza entrambe**, scegliendo caso per caso la tecnica più adatta alle esigenze di accoppiamento, resilienza e latenza di ogni interazione. La regola guida è: **sincrono quando devo sapere il risultato, asincrono quando devo solo comunicare che qualcosa è successo.**